Secure AI Output Handling

OAuth 2.0 · RBAC · PII Redaction · Schema Validation

Project explained in simple terms: components, workflow and architecture

> **The project statement**
> "Achieved zero data-exposure incidents across security reviews by implementing OAuth 2.0 authentication, RBAC access control, PII redaction, and schema validation for safe handling of model outputs."

Contents

1. Overall Summary (In Simple Terms)

Key Terms Glossary

2. Components Explained in Detail

1. User / Client App
2. Identity Provider (OAuth 2.0 Authorization Server)
3. API Gateway (Token Checker)
4. RBAC Policy Engine + Role & Permission Store
5. Input PII Redaction + Redaction Token Vault
6. AI Model (LLM)
7. Output Schema Validator + Schema Registry
8. Output PII Redaction (Final Check)
9. Audit Logging & Monitoring
10. Safe Response

3. End-to-End Workflow

3.1 Happy Path

3.2 Each Stage in Plain Words

3.3 What Happens When Something Fails

3.4 Worked Example

4. Architecture Diagram

4.1 How to Read the Diagram (Legend)

4.2 Walking Through the Diagram, Box by Box

4.3 Trust Boundaries

5. Why This Achieved Zero Data-Exposure Incidents

1. Overall Summary (In Simple Terms)

This project built a secure “safety pipeline” around an AI model. Whenever someone asks the AI something, the request passes through a series of security checkpoints before the AI sees it, and the AI’s answer passes through more checkpoints before anyone sees it.

Think of it like a secure bank vault with four layers of protection:

- **OAuth 2.0 (Who are you?):** Every user must prove their identity and receive a short-lived digital pass (token). No pass, no entry.
- **RBAC (What are you allowed to do?):** Each person has a role, like “analyst” or “admin”. The role decides exactly what they can ask and see.
- **PII Redaction (Hide personal details):** Names, emails, phone numbers and similar data are automatically masked before reaching the AI, and checked again in the AI’s answer.
- **Schema Validation (Is the answer safe and well-formed?):** The AI’s answer must match an approved template exactly. Anything unexpected is blocked.

> **The result**
>
> Because every request and answer passes through all four layers, no personal or sensitive data was exposed. Security reviewers tested the system and found zero data-exposure incidents.
>
> The key idea is “defence in depth”: if one layer ever missed something, the next one would catch it.

Key Terms Glossary

Term	Simple meaning
OAuth 2.0	An industry-standard way to log in and get a temporary access pass without sharing your password with every app.
Access token (JWT)	A signed digital pass that says who you are, what you may do and when the pass expires.
Identity Provider	The trusted login service that checks identity and issues tokens.
API Gateway	The single front door that checks every request’s pass.
RBAC	Role-Based Access Control: permissions are given to roles, and people are given roles.
PII	Personally Identifiable Information: anything that identifies a person (name, email, phone, ID numbers).
Redaction	Hiding or replacing sensitive data with placeholders like [EMAIL_1].

Term	Simple meaning
LLM	Large Language Model: the AI that reads text and writes answers.
JSON Schema	A rulebook describing exactly what a valid answer must look like.
401 / 403	401 = not logged in or invalid pass. 403 = logged in but not allowed.
Defence in depth	Several independent security layers, so one failure does not cause a breach.

2. Components Explained in Detail

Below, every part of the system is explained end to end: what it is, a simple analogy, exactly how it works step by step, the small details that matter, and what it protects against.

1. User / Client App

In one line: The website, mobile app or internal service that sends a question or task to the AI.

Simple analogy: *The customer walking up to a bank counter.*

How it works, step by step

1. The user opens the app and clicks “Sign in”.
2. The app sends the user to the Identity Provider’s login page (the app never sees the password).
3. After login, the app receives an access token and keeps it in memory.
4. For every AI request, the app attaches the token in the request header: “Authorization: Bearer <token>”.
5. When the safe answer comes back, the app shows it on screen.

Small details that matter

- **Never stores passwords:** Login happens on the Identity Provider’s page, not inside the app.
- **HTTPS only:** All traffic is encrypted in transit (TLS), so nobody on the network can read it.
- **Token storage:** Tokens are kept in memory or secure, HTTP-only cookies rather than in places scripts can easily read.
- **No hidden trust:** The app is treated as untrusted; every check happens again on the server side.

What it protects against: Password theft through the app, and anyone listening on the network.

2. Identity Provider (OAuth 2.0 Authorization Server)

In one line: The trusted service that checks who you are and hands out short-lived “access passes” (tokens).

Simple analogy: *The front desk that checks your ID card and gives you a visitor badge that expires at the end of the day.*

How it works, step by step

1. Receives the login request from the app, using the Authorization Code flow with PKCE (a one-time secret that proves the same app that started the login is the one finishing it).
2. Verifies the user’s password and, where enabled, multi-factor authentication (MFA).
3. Returns a one-time authorization code to the app.

4. The app swaps that code for an access token (and usually a refresh token).
5. The access token is a signed JWT containing: who the user is (subject), who issued it (issuer), who it is for (audience), what it may do (scopes), the user's roles, and an expiry time.
6. Publishes its public signing keys (a JWKS endpoint) so other services can check token signatures without calling back every time.

Small details that matter

- **Short lifetime:** Access tokens expire quickly (for example 15–60 minutes), so a stolen token stops working soon.
- **Refresh tokens:** Let the app get a new access token without asking the user to log in again; they can be revoked centrally.
- **Scopes:** Limit a token to specific actions, such as "ai:query" but not "ai:admin".
- **Machine clients:** Back-end services use the Client Credentials flow (their own ID and secret) instead of a person logging in.
- **Digital signature:** Any change to the token makes the signature invalid, so tokens cannot be forged or edited.

What it protects against: Fake users, forged or edited tokens, and long-lived stolen credentials.

3. API Gateway (Token Checker)

In one line: The single front door every request must pass through; it rejects anyone without a valid pass.

Simple analogy: *The security guard at the building entrance scanning every badge.*

How it works, step by step

1. Receives the request over HTTPS.
2. Reads the token from the Authorization header. No token means immediate rejection.
3. Checks the signature using the Identity Provider's public key.
4. Checks the expiry time (exp), the issuer (iss), the audience (aud) and the required scope.
5. Applies rate limits so one user cannot flood the system.
6. If everything passes, forwards the request with the verified user ID and roles attached. Otherwise, returns 401 Unauthorized.

Small details that matter

- **401 vs 403:** 401 means "we don't know who you are"; 403 (used later) means "we know you, but you are not allowed".
- **Key caching:** Public keys are cached and refreshed on rotation, so checks stay fast.
- **Request size limits:** Very large or malformed inputs are rejected before reaching the AI.

- **Single entry point:** The AI model has no public address; the gateway is the only way in.

What it protects against: Anonymous access, expired or tampered tokens, and request flooding.

4. RBAC Policy Engine + Role & Permission Store

In one line: Decides what a verified user is allowed to do, based on their role.

Simple analogy: *A hotel key card: guests open their own room, cleaners open every room on their floor, managers open the office.*

How it works, step by step

1. Takes the user's roles from the verified token (for example "analyst", "support_agent", "admin").
2. Looks up what each role is allowed to do in the Role & Permission Store (for example "support_agent can ask the AI about tickets, but cannot see full payment data").
3. Compares the requested action and data with those permissions.
4. Default is deny: if no rule clearly allows it, the request is blocked with 403 Forbidden.
5. If allowed, records the user's data-visibility level, which is used again at the end to decide what the answer may show.

Small details that matter

- **User → Role → Permission:** Users are given roles; roles hold permissions. You never give permissions to a person directly, which keeps things tidy and auditable.
- **Least privilege:** Each role gets only the minimum it needs.
- **Field-level rules:** Roles can control not just which tools are used but which fields (such as salary or phone number) may appear.
- **Central policies:** Changing a role in one place updates access everywhere instantly.
- **Regular reviews:** Role assignments are reviewed so people who change jobs lose old access.

What it protects against: Employees or services seeing or doing more than their job needs (privilege abuse).

5. Input PII Redaction + Redaction Token Vault

In one line: Finds personal information in the user's text and swaps it for placeholders before the AI sees it.

Simple analogy: *Blacking out names and numbers on a document before handing it to an outside contractor.*

How it works, step by step

1. Scans the incoming prompt for PII (Personally Identifiable Information): names, emails, phone numbers, addresses, ID numbers, card numbers, dates of birth and so on.

2. Uses three detection methods together: patterns (regular expressions for emails and phone numbers), checksum tests (such as the Luhn check for card numbers) and named-entity recognition (an NLP model that spots names, places and organisations).
3. Replaces each item with a typed placeholder, for example “John Smith” → [PERSON_1] and “john@x.com” → [EMAIL_1].
4. Stores the mapping (placeholder ↔ real value) in an encrypted vault, linked to this one request and set to expire.
5. Sends only the cleaned text onward to the AI model.

Small details that matter

- **Consistent placeholders:** The same person gets the same tag within one request, so the AI can still reason (“[PERSON_1] emailed [PERSON_2]”).
- **Encrypted vault:** Mappings are encrypted and only readable by the redaction service; they are auto-deleted after a short time.
- **Confidence threshold:** When unsure, it redacts (better safe than sorry).
- **Configurable list:** Teams can add company-specific identifiers such as employee or account numbers.

What it protects against: Personal data leaking to the AI model, the model provider, or logs.

6. AI Model (LLM)

In one line: The large language model that reads the cleaned request and writes an answer.

Simple analogy: *A clever contractor who only ever receives blacked-out documents.*

How it works, step by step

1. Receives the redacted prompt plus a system instruction that says to answer in a fixed JSON format.
2. Generates its answer.
3. Returns the raw output, which is treated as untrusted until checked.

Small details that matter

- **No direct access:** The model is never reachable from the internet; only the pipeline can call it.
- **Structured output:** It is asked for JSON in an agreed shape so the next step can check it automatically.
- **Untrusted by design:** Because models can make mistakes or be tricked by clever inputs (prompt injection), their output is always verified.

What it protects against: Being the place where sensitive data accumulates, since it only ever sees redacted text.

7. Output Schema Validator + Schema Registry

In one line: Checks the AI's answer matches an approved template exactly before it goes any further.

Simple analogy: *A form-checker who rejects any form with missing boxes, wrong formats or extra scribbles in the margins.*

How it works, step by step

1. Fetches the approved JSON Schema for this type of request from the Schema Registry.
2. Checks the output is valid JSON at all.
3. Checks every required field is present and has the correct type (text, number, true/false, list).
4. Checks values are allowed: enums (only "low", "medium" or "high"), number ranges, text length limits and formats (dates, IDs).
5. Rejects any extra fields that are not in the schema (additionalProperties = false), which stops unexpected data from slipping through.
6. If it fails, asks the model to try again (limited retries); if it still fails, returns a safe error instead of the bad answer.

Small details that matter

- **Versioned schemas:** Each schema has a version, so changes are controlled and reviewed.
- **Strict mode:** Nothing outside the template is allowed.
- **Safe failure:** The user gets a clean error, never broken or half-checked output.
- **Downstream safety:** Other systems that consume the answer can trust its shape, preventing crashes and injection into databases or web pages.

What it protects against: Malformed, unexpected or manipulated AI output reaching users or other systems.

8. Output PII Redaction (Final Check)

In one line: A last scan of the validated answer to catch any personal data before it leaves.

Simple analogy: *A final inspection at the exit door before the package is shipped.*

How it works, step by step

1. Scans the validated answer again with the same PII detectors.
2. Masks any new personal data the AI may have produced or guessed.
3. Checks the user's visibility level from the RBAC step.
4. Only if the role allows, swaps placeholders back to real values using the vault (re-identification). Otherwise, values stay masked, such as "****1234".
5. Passes the final clean answer on.

Small details that matter

- **Two-sided redaction:** Checking both input and output gives two layers of protection.
- **Partial masking:** Some roles may see only the last four digits of a number.
- **Logged decisions:** Every re-identification is logged for audits.

What it protects against: The AI echoing, inventing or reconstructing personal data in its answer.

9. Audit Logging & Monitoring

In one line: Records what happened at every step (without storing personal data) and raises alerts when something looks wrong.

Simple analogy: *The CCTV and visitor logbook of the building.*

How it works, step by step

1. Every component writes an event: who, what, when, which decision (allow/deny), and a request ID linking the steps.
2. Logs themselves are redacted, so no raw PII is stored.
3. Logs go to tamper-resistant storage with restricted access.
4. Dashboards and alerts watch for spikes in 401/403 errors, repeated schema failures or unusual PII detections.

Small details that matter

- **Correlation ID:** One ID follows a request through all steps, making investigations quick.
- **Retention rules:** Logs are kept for a set period to meet security review and compliance needs.
- **Evidence for reviews:** These logs proved to security reviewers that every control worked.

What it protects against: Undetected attacks and having no evidence during an investigation or review.

10. Safe Response

In one line: The final answer delivered back to the user: authenticated, authorised, redacted and validated.

Simple analogy: *The sealed, checked parcel handed back to the customer.*

How it works, step by step

1. Returned through the gateway over HTTPS.
2. Contains only fields from the approved schema and only data the user's role may see.

Small details that matter

- **Consistent format:** The app always knows exactly what it will receive.

- **No surprises:** Nothing extra or personal slips through.

What it protects against: Data exposure at the very last step.

3. End-to-End Workflow

This is the full journey of one request, from the moment a user logs in until they see the answer. Step numbers match the red numbered circles in the architecture diagram (Section 4).

3.1 Happy Path (Everything Is Allowed)

Step	What happens	Simple explanation
1	Login request	User clicks Sign in; the app redirects to the Identity Provider.
2	Token issued	User logs in (with MFA). The app receives a short-lived signed access token.
3	AI request sent	App sends the question to the API Gateway with the token attached, over HTTPS.
4	Token verified	Gateway checks signature (using the provider's public key), expiry, issuer, audience and scope. Fail → 401.
5	Passed to RBAC	Gateway forwards the request with the verified user ID and roles.
6	Permission check	RBAC engine looks up role permissions in the store. Not allowed → 403. Allowed → visibility level noted.
7	Sent for cleaning	Request moves to Input PII Redaction.
8	PII masked	Personal data is replaced with placeholders; mapping saved in the encrypted vault.
9	AI called	Clean prompt plus formatting instructions sent to the AI model.
10	Answer generated	Model returns raw JSON output (still untrusted).
11	Schema check	Validator compares output to the approved schema from the registry. Fail → retry, then safe error.
12	Final PII scan	Output is scanned again; placeholders restored only if the role allows.
13	Safe response built	Clean, validated, role-filtered answer is prepared.
14	Returned to user	Answer travels back through the gateway to the app. Every step has been logged.

3.2 Each Stage in Plain Words

Stage A: Getting in (Steps 1–4)

The user proves who they are once and gets a temporary pass. On every request, the gateway checks the pass is real, still valid and meant for this system. Fake, expired or altered passes are turned away with a 401 error.

Stage B: Checking permission (Steps 5–6)

Knowing who you are is not enough. The RBAC engine checks whether your role lets you do this specific thing. If not, you get a 403 error and the AI is never called. If yes, the system notes how much data you are allowed to see.

Stage C: Cleaning the request (Steps 7–8)

Before the AI sees anything, personal details are swapped for placeholders. The real values are locked in an encrypted vault for a short time, so they can be restored later only for people allowed to see them.

Stage D: Asking the AI (Steps 9–10)

The AI receives only the cleaned text and instructions to answer in a fixed format. Its answer is treated as untrusted until checked.

Stage E: Checking the answer (Steps 11–12)

First, the answer must match the approved template exactly, with no missing, wrong or extra fields. Second, it is scanned again for personal data. Placeholders are only turned back into real values if the user's role allows it.

Stage F: Delivering the answer (Steps 13–14)

The final safe answer goes back to the user through the gateway. Every step along the way has been logged without storing personal data.

3.3 What Happens When Something Fails

Where it fails	Example	What the system does
Token check (Step 4)	Token is expired, forged or missing.	Returns 401 Unauthorized. Request stops. Event is logged.
RBAC check (Step 6)	Support agent asks for payroll data.	Returns 403 Forbidden. AI is never called. Event is logged.
Input redaction (Step 8)	Detector is unsure whether text is a name.	Redacts it anyway (fail-safe).
Schema check (Step 11)	AI returns an extra field or wrong data type.	Asks the AI to retry; after the retry limit, returns a safe error message.

Where it fails	Example	What the system does
Output redaction (Step 12)	AI writes an email address in its answer.	Masks it before sending; logs the detection for monitoring.
Any step	Unusual spike of failures.	Monitoring raises an alert for the security team.

3.4 Worked Example

Request: A support agent types: “Summarise the complaint from John Smith, john.smith@mail.com, phone 555-123-4567.”

- Steps 1–4: The agent is logged in; the token is valid and has the “ai:query” scope.
- Steps 5–6: Role “support_agent” may summarise tickets, but may only see masked contact details.
- Steps 7–8: Text becomes “Summarise the complaint from [PERSON_1], [EMAIL_1], phone [PHONE_1].”
- Steps 9–10: The AI returns {"summary": "[PERSON_1] reports a delayed refund…", "priority": "high"}.
- Step 11: The output matches the schema (summary is text, priority is one of low/medium/high, no extra fields).
- Step 12: The role allows the name but not full contact details, so [PERSON_1] becomes “John Smith” and any email stays masked.
- Steps 13–14: The agent sees a clean summary. No raw email or phone number ever reached the AI or the logs.

4. Architecture Diagram

The diagram below shows every component, how they connect, and the order of steps. Read it from top to bottom.

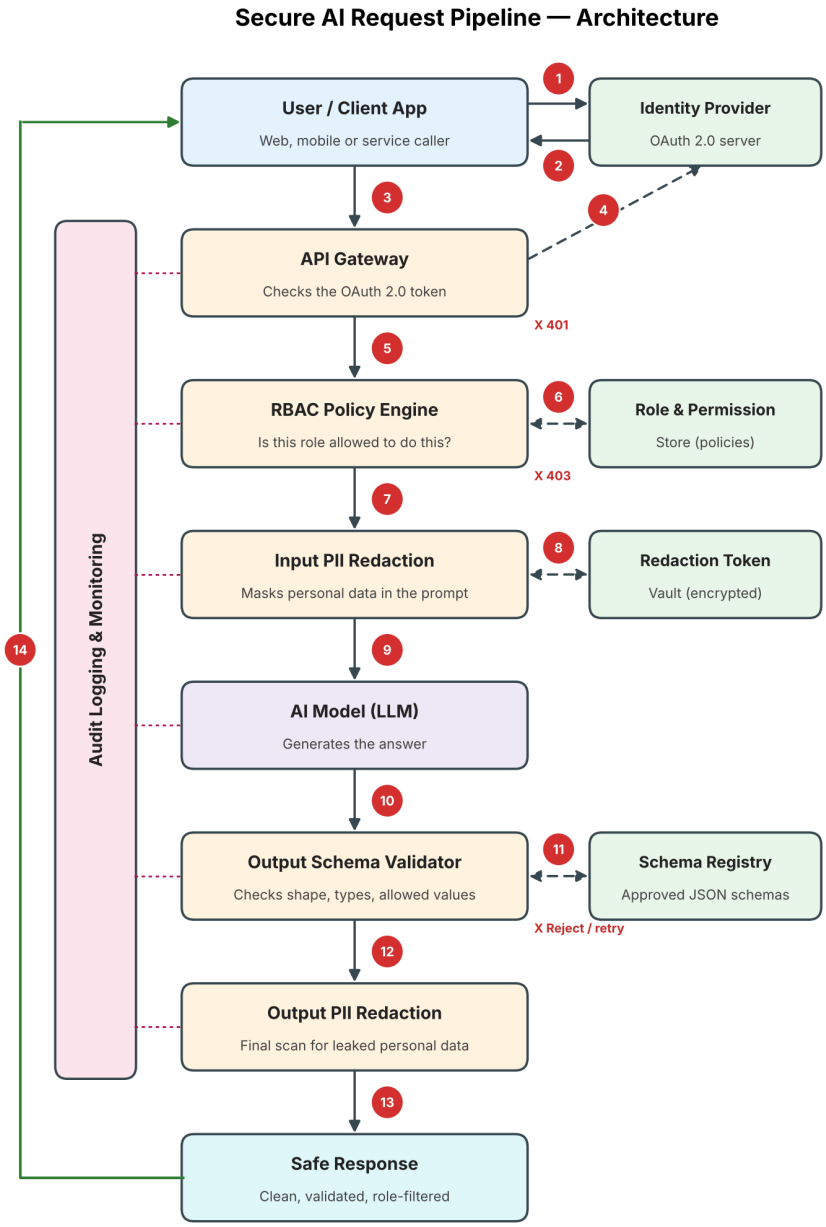


Figure 1: Secure AI request pipeline

4.1 How to Read the Diagram (Legend)

Diagram element	Meaning
Blue box	The user or client app (where requests start and answers end).
Orange boxes	Security checkpoints in the main path (gateway, RBAC, redaction, validation).
Purple box	The AI model itself.
Green boxes (right side)	Supporting services and data stores that checkpoints consult.
Light-cyan box	The final safe response.
Pink vertical bar (left)	Audit logging and monitoring, connected to every checkpoint.
Solid dark arrow	The main flow of a request from one step to the next.
Dashed dark arrow	A lookup to a supporting service (two-headed means ask and get an answer).
Dotted pink line	A checkpoint writing an event to the audit log.
Solid green line	The safe answer travelling back to the user.
Red numbered circle	Step number; matches the workflow table in Section 3.
Red “X” label	Where a request can be rejected (401, 403 or reject/retry).

4.2 Walking Through the Diagram, Box by Box

Top row: identity

User / Client App sends the user to the Identity Provider (arrow 1) and receives a token back (arrow 2). This happens once per login session, not on every request.

Front door: API Gateway

Arrow 3 carries every AI request with its token into the gateway. Dashed arrow 4 shows the gateway using the Identity Provider’s published public keys to verify the token. The “X 401” label marks where invalid tokens are rejected.

Permission layer: RBAC Policy Engine

Arrow 5 moves verified requests to the RBAC engine. Dashed two-way arrow 6 shows it asking the Role & Permission Store what this role can do. “X 403” marks where disallowed requests stop.

Privacy layer (input): Input PII Redaction

Arrow 7 delivers allowed requests for cleaning. Dashed two-way arrow 8 shows placeholder mappings being saved to (and later read from) the encrypted Redaction Token Vault.

Intelligence: AI Model

Arrow 9 sends only the cleaned prompt to the model. The model has no connections to the outside world or to any data store, which is why it has no green box beside it.

Quality layer: Output Schema Validator

Arrow 10 carries the raw answer to the validator. Dashed two-way arrow 11 shows it fetching the approved schema from the Schema Registry. “X Reject / retry” marks where bad output is stopped.

Privacy layer (output): Output PII Redaction

Arrow 12 sends the validated answer for a final privacy scan. It reuses the vault mappings and the role’s visibility level to decide what to unmask.

Delivery: Safe Response

Arrow 13 produces the safe response. Green line 14 shows it returning to the user (in practice, back through the gateway over HTTPS).

Left side: Audit Logging & Monitoring

Dotted pink lines link every checkpoint to the audit bar. Each decision is recorded, without personal data, so security reviewers can trace any request from start to finish.

4.3 Trust Boundaries

Zone	What is in it	Trust level
Outside	User device, client app	Untrusted: everything is re-checked on the server.
Edge	API Gateway	Trust is established here by verifying the token.
Control zone	RBAC, redaction, validator, registries, vault	Trusted internal services, reachable only from the gateway.
Model zone	AI model	Isolated; sees only redacted data; its output is treated as untrusted.
Evidence zone	Audit logs and monitoring	Write-only for services; read only by the security team.

5. Why This Achieved Zero Data-Exposure Incidents

Security reviews typically test whether data can leak through any of these routes. Each route is blocked by at least one layer, and most by two or more.

Possible leak route	Layer(s) that block it
Someone uses the system without logging in	OAuth 2.0 tokens + gateway verification
Stolen or forged token	Signature check, short expiry, scopes, revocable refresh tokens
Logged-in user sees data beyond their job	RBAC deny-by-default + field-level visibility in output redaction
Personal data sent to the AI model	Input PII redaction + encrypted vault
AI repeats or invents personal data	Output PII redaction
AI returns unexpected or manipulated content	Strict schema validation (no extra fields, typed values, retries)
Personal data stored in logs	Redacted audit logging
Attack goes unnoticed	Monitoring and alerts on failures and PII detections

Key takeaway

Each control is simple on its own. The strength comes from stacking them: identity first, then permission, then privacy on the way in, then format and privacy on the way out, with everything logged.

That layered design is what allowed the system to pass security reviews with zero data-exposure incidents.